

SIMATS ENGINEERING

ASSESSMENT TOOL 1

NAME: SIVARANGJENII B

REG NO:192521375

COURSE NAME: COMPUTER NETWORKS

COURSE CODE: CSA0703

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards

(BL2, BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.
2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate

the correctness of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.
4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.
5. Design and implement a Python class named `StarSwitch` that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

1. Star Topology Simulation

```
[1] 1m
# Star Topology Simulation

n = int(input("Enter number of hosts: "))

switch = "Switch1"

print("\nStar Topology")
print("-----")

for i in range(1, n + 1):
    length = int(input(f"Enter Cat6 cable length for Host{i} (meters): "))
    print(f"Host{i} ----({length}m Cat6 UTP)----> {switch}")

print("\nVerification")
print(f"All hosts are connected through the central switch.")

... Enter number of hosts: 4

Star Topology
-----
Enter Cat6 cable length for Host1 (meters): 20
Host1 ----(20m Cat6 UTP)----> Switch1
Enter Cat6 cable length for Host2 (meters): 30
Host2 ----(30m Cat6 UTP)----> Switch1
Enter Cat6 cable length for Host3 (meters): 15
Host3 ----(15m Cat6 UTP)----> Switch1
Enter Cat6 cable length for Host4 (meters): 45
Host4 ----(45m Cat6 UTP)----> Switch1

Verification
All hosts are connected through the central switch.
```

2. IEEE 802.3 Cable Validation

```
2] 0s
def validate_segment(length_m):
    if length_m <= 100:
        return True
    else:
        print("Error: Cable length exceeds IEEE 802.3 limit (100 meters).")
        return False

# Test Cases
print("Test Case 1:", validate_segment(50))
print("Test Case 2:", validate_segment(100))
print("Test Case 3:", validate_segment(120))

... Test Case 1: True
Test Case 2: True
Error: Cable length exceeds IEEE 802.3 limit (100 meters).
Test Case 3: False
```

4. Client-Server Socket Program

```
[3] 38s
import socket
import threading
import time

# Simulated MAC Address Table
mac_table = {}

# ----- SERVER -----
def server():
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind(("localhost", 5000))
    server_socket.listen(1)

    print("Switch (Server) is waiting for connection...\n")

    conn, addr = server_socket.accept()
    print(f"Host Connected:", addr)

    data = conn.recv(1024).decode()

    # Data format: MAC,FRAME
    mac, frame = data.split(",")

    # MAC Learning
    mac_table[mac] = "Port1"

    print("\nFrame Received:", frame)

    print("\nMAC Address Table")
    print("-----")
    for mac_addr, port in mac_table.items():
        print(mac_addr, " --> ", port)

    print("\nFrame Forwarded Successfully.")

    conn.send("Frame Received and Forwarded".encode())

    conn.close()
    server_socket.close()

# ----- CLIENT -----
def client():
    time.sleep(1) # Wait until server starts

    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
[3] 38s
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(("localhost", 5000))

mac = input("Enter MAC Address: ")
frame = input("Enter Frame: ")

client_socket.send(f"{mac},{frame}".encode())

reply = client_socket.recv(1024).decode()
print("\nServer:", reply)

client_socket.close()

# Run Server and Client Together
server_thread = threading.Thread(target=server)
client_thread = threading.Thread(target=client)

server_thread.start()
client_thread.start()

server_thread.join()
client_thread.join()

... Switch (Server) is waiting for connection...

Host Connected: ('127.0.0.1', 40078)
Enter MAC Address: AA:BB:CC:DD:EE:01
Enter Frame: Hello Switch

Frame Received: Hello Switch

MAC Address Table
-----
AA:BB:CC:DD:EE:01 --> Port1

Frame Forwarded Successfully.

Server: Frame Received and Forwarded
```

5. Star Switch Class

```
[4] ✓ 0s class StarSwitch:

    def __init__(self):
        self.ports = []

    def add_port(self, host_id):
        self.ports.append(host_id)
        print(host_id, "connected.")

    def remove_port(self, host_id):
        if host_id in self.ports:
            self.ports.remove(host_id)
            print(host_id, "removed.")

    def broadcast(self, source):

        print("\nBroadcast from", source)

        for host in self.ports:
            if host != source:
                print(source, "--->", host)

# Driver Program

sw = StarSwitch()

sw.add_port("Host1")
sw.add_port("Host2")
sw.add_port("Host3")
sw.add_port("Host4")

print("\nActive Ports:", sw.ports)

sw.broadcast("Host1")

sw.remove_port("Host3")

print("\nActive Ports:", sw.ports)

sw.broadcast("Host2")
```

```
... Host1 connected.
Host2 connected.
Host3 connected.
Host4 connected.

Active Ports: ['Host1', 'Host2', 'Host3', 'Host4']

Broadcast from Host1
Host1 ---> Host2
Host1 ---> Host3
Host1 ---> Host4
Host3 removed.

Active Ports: ['Host1', 'Host2', 'Host4']

Broadcast from Host2
Host2 ---> Host1
Host2 ---> Host4
```